Following questions provide a strong technical basis for auditing MySQL databases using Linux CLI tools, covering user privileges, SQL injection detection, authentication methods, log monitoring, and hardening techniques.

Question 1:
During a recent database audit, you discovered that MySQL logs, including general query logs and error logs, are stored in a publicly accessible directory with weak file permissions. Discuss the security implications of this misconfiguration and propose corrective measures to protect MySQL log integrity and confidentiality.

Solution:

Security Implications:

1. **Exposure of Sensitive Data:**
   - MySQL logs may contain sensitive information such as user credentials (if logging is not properly configured) and SQL queries, including potential PII (Personally Identifiable Information).
   - If logs are publicly accessible, attackers can extract valuable information for privilege escalation and reconnaissance.
2. **Log Tampering and Integrity Risks:**
   - If unauthorized users have write access, they can modify or delete logs to cover their tracks after exploiting a vulnerability.
   - Loss of audit trails impacts forensic investigations and compliance with regulations like GDPR, HIPAA, and PCI DSS.
3. **Privilege Escalation:**
   - If the MySQL process runs with excessive privileges (e.g., as root or another privileged user), an attacker gaining access to logs might exploit misconfigurations to escalate privileges.

Corrective Measures:

1. **Restrict File Permissions:**
   - Modify log file permissions to ensure that only the MySQL service user (`mysql`) can read or write logs. Use the following command:
     ```
     chown mysql:mysql /var/log/mysql/*.log
     chmod 640 /var/log/mysql/*.log
     ```
   - This prevents unauthorized users from accessing or modifying log files.
2. **Store Logs in a Secure Directory:**
   - Configure MySQL to store logs in a restricted directory, such as `/var/log/mysql/`, which is not accessible to public users:
     ```
     [mysqld]
     log-error=/var/log/mysql/error.log
     general-log-file=/var/log/mysql/general.log
     ```
   - Avoid storing logs in web-accessible directories like `/var/www/html/`.
3. **Enable Log Rotation and Encryption:**
   - Use `logrotate` to automatically archive and remove old logs, preventing unauthorized access to outdated sensitive data. Example `logrotate` configuration:
     ```
     /var/log/mysql/*.log {
       daily
       rotate 7
       compress
       missingok
       notifempty
       create 640 mysql mysql
     }
     ```
   - Encrypt logs using file system encryption (`LUKS`, `eCryptfs`) or MySQL Enterprise Audit Plugin to prevent unauthorized reading.
4. **Monitor and Restrict Access to Logs:**

- Regularly audit log file access using Linux tools like `auditd`:
  `auditctl -w /var/log/mysql/ -p rwxa -k mysql_logs`
- Implement SIEM (Security Information and Event Management) solutions to detect and alert on suspicious log access.

By implementing these controls, organizations can protect MySQL logs from unauthorized access, maintain log integrity, and ensure compliance with security best practices.

Question 2: Assessing User Privileges in MySQL

Scenario: You are auditing a MySQL database on a Linux server. One of your objectives is to ensure that user privileges are appropriately assigned to prevent unauthorized data access or modification.

Task: Using the MySQL CLI, detail the steps to review and assess the privileges granted to all users. Identify potential security risks associated with misconfigured privileges and suggest remediation measures.

Solution:

1. **Access the MySQL CLI:**
   `mysql -u root -p`
   Enter the root password when prompted.
2. **List All Users and Their Hosts:**
   `SELECT User, Host FROM mysql.user;`
   This command retrieves all users and their corresponding host information.
3. **Review User Privileges:**
   `SHOW GRANTS FOR 'username'@'host';`
   Replace `'username'` and `'host'` with the specific user and host values obtained from the previous step. Repeat for each user.
4. **Identify Potential Security Risks:**
   - **Excessive Privileges:** Users with `ALL PRIVILEGES` may have more access than necessary, increasing the risk of unauthorized data manipulation.
   - **Wildcard Hosts:** Entries like `'%'` in the Host field allow access from any IP address, potentially exposing the database to external threats.
   - **Anonymous Users:** Presence of users with empty `User` fields can be a significant security concern.
5. **Remediation Measures:**
   - **Principle of Least Privilege:** Grant users only the permissions essential for their tasks. For example, use `SELECT`, `INSERT`, `UPDATE` instead of `ALL PRIVILEGES` when full access is unnecessary.
     `GRANT SELECT, INSERT, UPDATE ON database_name.* TO 'username'@'host';`
   - **Restrict Host Access:** Replace `'%'` with specific IP addresses or hostnames to limit access.
   - **Remove Anonymous Users:**
     ```
     DELETE FROM mysql.user WHERE User='';
     FLUSH PRIVILEGES;
     ```
     This command removes anonymous user accounts and refreshes privileges.

Regularly reviewing and adjusting user privileges helps maintain a secure MySQL environment by minimizing unauthorized access risks.

Question 3: Detecting and Mitigating SQL Injection Vulnerabilities

Scenario: During a security audit of a MySQL database running on a Linux server, you suspect that certain applications interacting with the database might be vulnerable to SQL injection attacks.

Task: Explain how to detect signs of SQL injection vulnerabilities using MySQL logs and Linux command-line tools. Additionally, outline strategies to mitigate these vulnerabilities.

Solution:

1. **Enable General Query Logging (if not already enabled):** Edit the MySQL configuration file (e.g., `/etc/mysql/my.cnf` or `/etc/my.cnf`) to include:

```
[mysqld]
general_log = 1
general_log_file = /var/log/mysql/general.log
```
Restart MySQL to apply changes:
```
sudo systemctl restart mysql
```

2. **Analyze the General Query Log:** Use command-line tools like `grep` and `awk` to search for suspicious patterns indicative of SQL injection:
```
grep -i "union select" /var/log/mysql/general.log
grep -i "or 1=1" /var/log/mysql/general.log
```
These commands search for common SQL injection patterns in the log file.
3. **Review Web Server Logs:** Examine web server logs (e.g., `/var/log/apache2/access.log`) for unusual query strings:
```
grep -i "union select" /var/log/apache2/access.log
grep -i "or 1=1" /var/log/apache2/access.log
```
4. **Mitigation Strategies:**
    - **Use Prepared Statements:** Implement parameterized queries in application code to prevent SQL injection. For example, in PHP using PDO:
```
$stmt = $pdo->prepare('SELECT * FROM users WHERE email = :email');
$stmt->execute(['email' => $email]);
```
    - **Input Validation:** Sanitize and validate all user inputs to ensure they conform to expected formats.
    - **Least Privilege Principle:** Ensure the database user account used by the application has only the necessary permissions.
    - **Regular Updates:** Keep the MySQL server and associated applications updated to patch known vulnerabilities.

By proactively monitoring logs and implementing secure coding practices, SQL injection risks can be significantly reduced.

Question 4: Auditing MySQL Authentication Methods

Scenario: As part of a comprehensive database audit on a Linux server, you need to verify that MySQL authentication methods are securely configured to prevent unauthorized access.

Task: Describe the steps to audit MySQL user authentication methods using the CLI. Highlight potential vulnerabilities associated with authentication mechanisms and recommend best practices for secure authentication.

Solution:

1. **Access the MySQL CLI:**
```
mysql -u root -p
```
2. **Review Authentication Methods:** For MySQL 5.7 and later:
Run the following SQL command to check which authentication plugin is used for each MySQL user:
```
SELECT user, host, plugin FROM mysql.user;
```
This command displays the authentication methods assigned to each user.
3. **Identify Potential Authentication Vulnerabilities:**
    - **Use of `mysql_native_password`:**
        - Older MySQL versions may rely on `mysql_native_password`, which is considered weaker than `caching_sha2_password`.
    - **Presence of Empty Passwords:**
```
SELECT user, host FROM mysql.user WHERE authentication_string = '';
```
        - If any users have an empty `authentication_string`, it indicates a serious security risk.
    - **Anonymous Users:**
```
SELECT user, host FROM mysql.user WHERE user = '';
```
        - Anonymous users should be removed to prevent unauthorized access.
4. **Best Practices for Secure Authentication:**

- **Use `caching_sha2_password`:**
  - This authentication plugin is more secure than `mysql_native_password` and should be used when possible.

```
ALTER USER 'username'@'host' IDENTIFIED WITH caching_sha2_password BY
'secure_password';
```

- **Enforce Strong Password Policies:**
  - Enable MySQL's `validate_password` plugin to enforce password complexity requirements:
    `INSTALL PLUGIN validate_password SONAME 'validate_password.so';`
    Configure the policy:
    `SET GLOBAL validate_password.policy = STRONG;`
- **Disable Remote Root Logins:**

```
UPDATE mysql.user SET host='localhost' WHERE user='root';
```

  - This ensures the root user can only authenticate from `localhost`.
- **Regularly Audit Users and Remove Unnecessary Accounts:**

```
DELETE FROM mysql.user WHERE user='old_user';
FLUSH PRIVILEGES;
```

- **Enable Multi-Factor Authentication (MFA) if Supported:**
  - Some MySQL versions support multi-factor authentication for added security.

By implementing these security measures, you can significantly reduce the risk of unauthorized access to MySQL databases.

Question 5: Auditing MySQL Logs for Anomalous Activities

Scenario:

You are conducting a security audit on a MySQL database running on a Linux server. Your objective is to identify suspicious activities such as unauthorized access attempts, excessive failed logins, and unusual query patterns.

Task:

Describe how to audit MySQL logs for anomalies using Linux CLI tools. Include commands to analyze logs, detect suspicious activities, and implement logging best practices.

Solution:

1. **Check if Logging is Enabled:**
   Ensure MySQL is configured to log queries and authentication attempts by checking `my.cnf` (or `mysqld.cnf`):

```
cat /etc/mysql/my.cnf | grep -E 'log_error|slow_query_log|general_log'
```

2. **Analyze the MySQL Error Log for Failed Login Attempts:**

```
grep -i "Access denied" /var/log/mysql/error.log
```

   - Look for multiple failed login attempts from unknown IPs, which could indicate brute-force attacks.
3. **Review the General Query Log for Unusual Activity:**

```
grep -E 'DROP|ALTER|DELETE' /var/log/mysql/general.log
```

   - Unexpected `DROP TABLE` or `DELETE` commands might indicate unauthorized modifications.
4. **Monitor the Slow Query Log for Performance and Security Issues:**

```
tail -f /var/log/mysql/slow.log
```

   - A sudden spike in slow queries could indicate SQL injection attempts.
5. **Logging Best Practices:**
   - **Enable Audit Logging (For MySQL Enterprise or MariaDB):**

```
INSTALL PLUGIN server_audit SONAME 'server_audit.so';
SET GLOBAL server_audit_logging = ON;
```

   - **Rotate Logs to Prevent Overgrowth:**

```
logrotate /etc/logrotate.d/mysql
```

   - **Send Logs to a Remote Security Information and Event Management (SIEM) System:**

```
rsyslog -f /etc/rsyslog.conf
```

By continuously monitoring MySQL logs and implementing best practices, you can detect and prevent security threats effectively.

Question 6: Hardening MySQL Server on Linux

Scenario:

Your organization wants to strengthen the security of its MySQL database server on Linux to protect against unauthorized access, data breaches, and misconfigurations.

Task:

Outline a step-by-step process to harden a MySQL database server running on Linux. Include security settings, network restrictions, and database-level controls.

Solution:

1. **Update MySQL and Linux System:**
   ```
   sudo apt update && sudo apt upgrade -y
   ```
   Keeping software up to date mitigates known vulnerabilities.
2. **Restrict Remote Root Logins:**
   ```
   UPDATE mysql.user SET host='localhost' WHERE user='root';
   FLUSH PRIVILEGES;
   ```
   - Prevents root login from external hosts.
3. **Disable Unused Network Services:**
   ```
   sudo systemctl disable --now mysql@bootstrap
   ```
   - Only keep the necessary MySQL services running.
4. **Change the Default MySQL Port:**
   - Edit `/etc/mysql/my.cnf`:
     ```
     [mysqld]
     port = 3307
     ```
   - Restart MySQL:
     ```
     sudo systemctl restart mysql
     ```
5. **Use a Firewall to Restrict Database Access:**
   ```
   sudo ufw allow from 192.168.1.100 to any port 3306
   sudo ufw enable
   ```
   - Allows MySQL connections only from a specific IP.
6. **Implement Secure Authentication:**
   ```
   ALTER USER 'admin'@'%' IDENTIFIED WITH caching_sha2_password BY
   'StrongPassword123!';
   ```
7. **Encrypt Data in Transit:**
   - Enable SSL by modifying `/etc/mysql/my.cnf`:
     ```
     [mysqld]
     require_secure_transport = ON
     ```
   - Restart MySQL:
     ```
     sudo systemctl restart mysql
     ```
8. **Regularly Audit and Monitor User Accounts:**
   ```
   SELECT user, host, authentication_string FROM mysql.user;
   ```
   - Identify unauthorized or weakly authenticated users.

By following these steps, the MySQL server becomes significantly more resilient against attacks.
